

11. Additional Techniques for Data Analytics

*“The core advantage of data is that
it tells you something about the world
that you didn't know before.”*
— Hilary Mason

This chapter rounds out the book with additional techniques for further research and development in data analytics: handling imbalanced data (where class sizes differ greatly, making it hard to learn an effective model), handling high-dimensional data (many attributes, requiring feature reduction for both model quality and processing speed), and analyzing unstructured data — text and images — as a starting point for further study.

11.1 Techniques for Imbalanced Data

Imbalanced datasets are a common machine-learning problem, where classes have very different numbers of examples: the smaller class is the minority class, the larger is the majority class. This hurts a model's ability to classify the minority class well, since it learns predominantly from the majority class and tends to default to it for new data. Over-sampling is a common technique to address this, increasing the minority class's size to better match the majority class.

Over-Sampling the Minority Class

Over-sampling increases the number of minority-class examples to approach the majority class's size, helping the model learn the minority class better. A popular technique is SMOTE (Synthetic Minority Over-sampling Technique), which randomly synthesizes new minority-class examples (Figure 11.1-1, panel 1) based on nearby existing examples (panel 2), following these steps:

1. Pair up nearby minority-class examples (e.g., the solid stars shown).
2. Randomly synthesize a new example between each pair (shown as translucent stars) — in practice, by taking the difference between each attribute pair, multiplying it by a random value between 0 and 1, and adding it to the original attribute value.
3. Repeat until the minority class approaches the majority class's size.

For example, given a pair $(x_1, x_2) = (1, 1)$ and $(2, 3)$, and a random value of 0.4: $x_1 = 1 - (1-2) \times 0.4 = 1.4$, and $x_2 = 1 - (1-3) \times 0.4 = 1.8$, giving a synthesized point of $(1.4, 1.8)$.

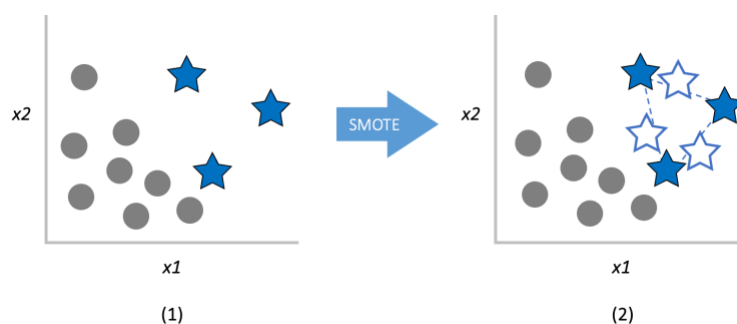


Figure 11.1-1: Over-sampling the minority class with SMOTE

Applying Over-Sampling

Using Scikit-Learn's SMOTE implementation:

```
import pandas as pd
from imblearn.over_sampling import SMOTE

CSV_PATH = 'https://rathachai.github.io/DA-LAB/datasets/imbalanced-
data.csv'
```

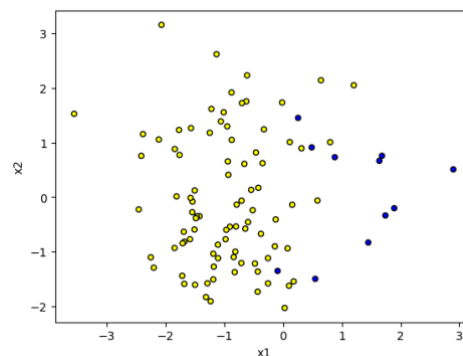
```
df = pd.read_csv(CSV_PATH)
df
```

This dataset has 100 rows and 3 columns (x1, x2, y), with y a class of yellow (89 rows) or blue (11 rows) — blue is the minority class.

	x1	x2	y
0	0.880028	0.730150	blue
1	-1.568666	1.263820	yellow
2	1.682727	0.756070	blue
3	-2.457948	-0.226560	yellow
4	-0.431319	-1.365023	yellow
...	...	...	...

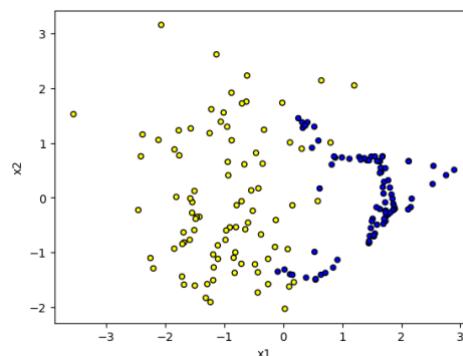
```
X = df[['x1', 'x2']]
y = df['y']
X.plot.scatter('x1', 'x2', c=y, edgecolors='black')
```

The scatter plot (colored by y, black-edged for clarity) shows far fewer blue points than yellow:



```
sm = SMOTE()
X_new, y_new = sm.fit_resample(X, y)
X_new.plot.scatter('x1', 'x2', c=y_new, edgecolors='black')
```

After SMOTE, both classes are roughly balanced, with new blue points clustered near the original ones:



11.2 Techniques for High-Dimensional Data

Real datasets can have dozens or hundreds of attributes, which is computationally expensive for cluster analysis and hard to visualize, since many dimension-pairs would need comparing. Dimensionality reduction is therefore a popular way to cut computational cost.

Principal Component Analysis (PCA)

PCA is the most popular dimensionality-reduction technique, keeping only the most important features.

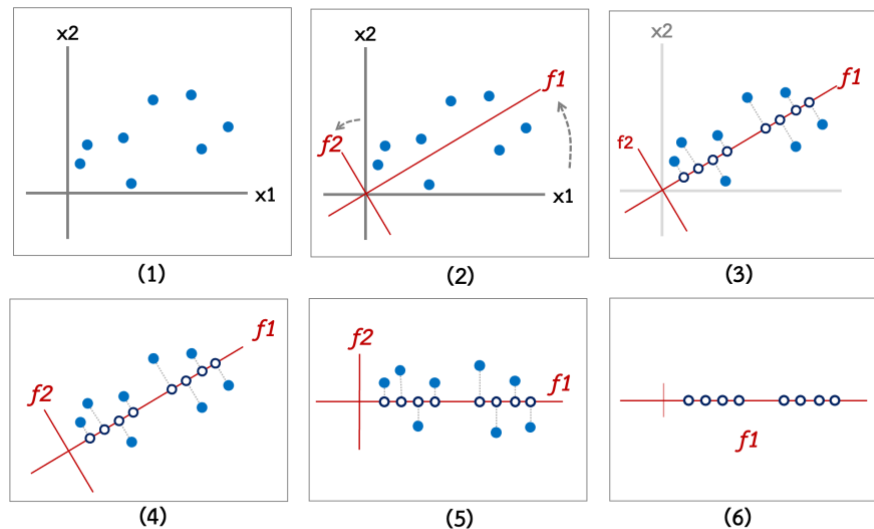


Figure 11.2-1: Steps in PCA dimensionality reduction

Example: Steps of PCA

To illustrate simply, here's how PCA reduces 2 dimensions to 1, following Figure 11.2-1:

4. Start with the original data, with attributes x_1 and x_2 (2 dimensions), shown as a scatter plot.
5. Rotate the x_1/x_2 axes into new axes, f_1 and f_2 , using eigenvectors to find the optimal rotation.
6. Re-plot the data's coordinates on the new f_1/f_2 axes (only f_1 is shown here for clarity).
7. Remove the original axes, keeping only f_1 and f_2 .
8. Rotate f_1/f_2 back to a standard horizontal/vertical layout for easier presentation.
9. Since PCA determines f_1 is the more important feature, keep only that dimension — the resulting reduced dataset is then used for further processing or presentation.

Calculation Steps

PCA's underlying math involves matrices, vectors, and eigenvectors. Starting with a 4×2 data matrix X representing 2-dimensional data, shown as a scatter plot in Figure 11.2-2:

$$X = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 5 & 4 \\ 6 & 5 \end{bmatrix}$$

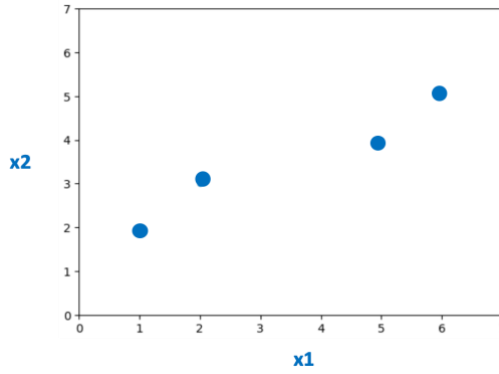


Figure 11.2-2: The original data before dimensionality reduction

Next, normalize X using $a_j^{(i)} = (x_j^{(i)} - \bar{x}_j) / \text{std}(x_j)$, where $x_j^{(i)}$ is the value at row i , column j ; $\bar{x}_j$ is column j 's mean; and $\text{std}(x_j)$ is column j 's standard deviation, giving matrix A :

$$A = \begin{bmatrix} -1.05 & -1.16 \\ -0.63 & -0.39 \\ 0.63 & 0.39 \\ 1.05 & 1.16 \end{bmatrix}$$

Then build the covariance matrix $C = (1/(m-1)) \cdot A \cdot A^T$, where m is the number of examples (here, 4):

$$C = \begin{bmatrix} 1 & 0.98 \\ 0.98 & 1 \end{bmatrix}$$

Next, find the eigenvalues λ by solving $(C - \lambda I)V = 0$, where I is the identity matrix and V is the eigenvector matrix:

$$\begin{bmatrix} 1 - \lambda & 0.98 \\ 0.98 & 1 - \lambda \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Solving the resulting quadratic equation $(1-\lambda)(1-\lambda) - (0.98 \times 0.98) = 0$, i.e., $\lambda^2 - 2\lambda + 1.96 = 0$, gives eigenvalues sorted from largest to smallest:

$$(1 - \lambda)(1 - \lambda) - (0.98 \times 0.98) = 0$$

$$\lambda^2 - 2\lambda + 1.96 = 0$$

$$\lambda = \begin{bmatrix} \lambda_1 \\ \lambda_2 \end{bmatrix} = \begin{bmatrix} 1.976 \\ 0.024 \end{bmatrix}$$

Substituting each λ back gives two eigenvectors, $\begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}$ and $\begin{bmatrix} -0.707 \\ 0.707 \end{bmatrix}$, forming eigenvector matrix E :

$$E = \begin{bmatrix} 0.707 & -0.707 \\ 0.707 & 0.707 \end{bmatrix}$$

E 's columns are ordered by importance, left to right. To reduce from 2 dimensions to 1, keep only the leftmost column, called E_{top1} :

$$E^{\text{top1}} = \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}$$

Finally, compute the reduced feature matrix $F = X \times E_{\text{top1}}$:

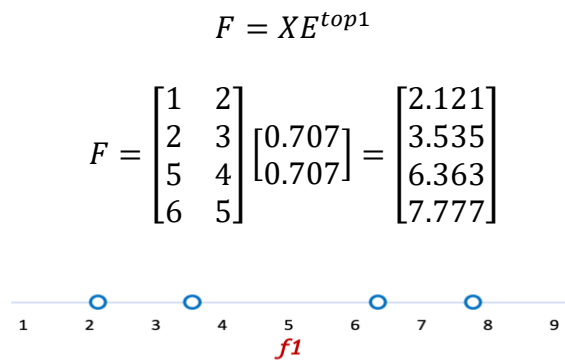


Figure 11.2-3: The data after PCA dimensionality reduction

Applying Dimensionality Reduction

Using the well-known Iris flower dataset — popular for classification and clustering — with 150 examples across 3 species (setosa, versicolor, virginica), each with 4 measured attributes: sepal length (SL), sepal width (SW), petal length (PL), and petal width (PW).

This example reduces 4 dimensions to 2, selecting the 4 attributes into DataFrame X, then applying PCA via Scikit-Learn with `n_components=2`, storing the result in F:

```
import pandas as pd
from sklearn.decomposition import PCA

csv_path = 'https://rathachai.github.io/DA-LAB/datasets/iris.csv'
df = pd.read_csv(csv_path)
X = df[['SL', 'SW', 'PL', 'PW']]
F = PCA(n_components=2).fit_transform(X)
```

	SL	SW	PL	PW	Species		f1	f2	Species
0	4.8	3.1	1.6	0.2	setosa	0	-2.59	-0.20	setosa
1	6.7	3.3	5.7	2.5	virginica	1	2.42	0.30	virginica
2	5.0	2.0	3.5	1.0	versicolor	2	-0.51	-1.26	versicolor
3	5.8	2.7	5.1	1.9	virginica	3	1.41	-0.57	virginica
4	6.9	3.1	5.4	2.1	virginica	4	2.11	0.37	virginica
...	...	...	...	...	...	...	...	...	...
145	5.8	2.7	3.9	1.2	versicolor	145	0.13	-0.31	versicolor
146	5.1	3.3	1.7	0.5	setosa	146	-2.30	0.11	setosa
147	7.6	3.0	6.6	2.1	virginica	147	3.40	0.55	virginica
148	7.3	2.9	6.3	1.8	virginica	148	2.93	0.35	virginica
149	5.7	2.5	5.0	2.0	virginica	149	1.34	-0.78	virginica

150 rows × 5 columns

(1)

(2)

Figure 11.2-4: Table view of the Iris flower dataset

Plotting the reduced data as a scatter plot, then styling points by species, shows the three species are largely separable by a straight line — suggesting these 2 PCA-reduced features could support a simple linear classifier, such as logistic regression.

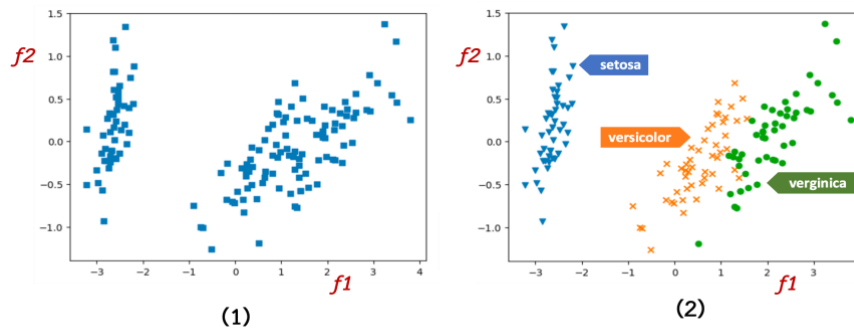


Figure 11.2-5: PCA-reduced Iris data by species

11.3 Text Data Analytics Techniques

Text analysis processes human-written text to find patterns, relationships, sentiment, and other usable insight — valuable in business, e.g., gauging sentiment toward a product or brand. It's a powerful tool for understanding text data to support better decisions, product development, and innovation.

Examples of text analysis include text classification (categorizing newsletters, news articles, or reviews), text summarization (condensing text without losing key meaning), machine translation, information retrieval (finding relevant information within large text volumes), and sentiment analysis (identifying whether a writer's tone is positive, negative, or neutral).

This section covers preparing text data and demonstrates sentiment analysis.

Preparing Text Data

Text such as “I love buying a data analytics book.” is, to a computer, just a string of characters with no inherent linguistic meaning. It must first be broken into meaningful units for analysis, using the techniques below.

Tokenization

Tokenization splits text into units called tokens — a fundamental step for computer understanding. English tokenization can simply split on spaces or punctuation; Thai is more challenging due to more complex sentence structure and many words with overlapping meaning. Approaches include splitting on spaces/punctuation, dictionary-based splitting, grammar rules (subject, verb, object, spelling, conjunctions, prepositions), or machine-learning language models.

Tokenizing “I love buying a data analytics book.” gives: ['I', 'love', 'buying', 'a', 'data', 'analytics', 'book', '.']

Removing Stop Words

Stop words are common natural-language words (a, an, the, this, that, those, I, you, he, she, it, is, am, are, etc.) that are grammatically necessary but usually add little analytical value, since they appear frequently across nearly all documents. Removing them saves processing resources and improves analysis quality.

Removing stop words (I, a, and the period) from the earlier example gives: ['love', 'buying', 'data', 'analytics', 'book']

Stemming and Lemmatization

These techniques normalize word variants to a common or equivalent form, easing analysis:

- **Stemming** — reduces a word to its root by trimming endings, using rules like removing “ing,” “ics,” or “ly,” or substituting endings (e.g., “iev”→“ief”). Here, “buying” becomes “buy,” and “analytics” becomes “analyt.”

- Lemmatization — converts a word to its standard dictionary form with equivalent meaning, typically using a lexical network (WordNet) or ontology, e.g., “better” → “good.”

Applying this to the earlier example gives: ['love', 'buy', 'data', 'analyt', 'book']

N-grams

Tokenization can sometimes split what should be one meaningful phrase into separate words (e.g., 'data', 'analyt'). N-grams address this by grouping n adjacent characters or words together — a 2-gram of these two words gives ['data analyt'].

Applying 2-grams to the earlier example gives: ['love buy', 'buy data', 'data analyt', 'analyt book'], which combined with the original tokens gives the full vocabulary: ['love', 'buy', 'data', 'analyt', 'book', 'love buy', 'buy data', 'data analyt', 'analyt book']

This can also produce phrases that aren't meaningful together, like “love buy” — usually rare enough to be filtered out in a later step.

Weighting Words by Importance (TF-IDF)

TF-IDF (Term Frequency–Inverse Document Frequency) measures a word's importance to a document, adjusting for the fact that some words are inherently more common. It's computed as $tf \times idf$:

- Term Frequency (TF) — how often a word appears in a document, typically normalized by the document's total word count. Frequent occurrence gives a high TF, suggesting importance to that document.

$$tf = \frac{\text{count of term within document}}{\text{total words within document}}$$

- Inverse Document Frequency (IDF) — the inverse of how many documents contain the word, typically dividing the corpus size by the count of documents containing the term, then applying a logarithm to dampen the term's weight. A word rare across the corpus gets a high IDF, suggesting it's distinctively important.

$$idf = \log \frac{\text{total documents within corpus}}{\text{documents containing the term}}$$

For example, computing TF-IDF for “analyt book” in a 9-word document ['love', 'buy', 'data', 'analyt', 'book', 'love buy', 'buy data', 'data analyt', 'analyt book'], assuming a 100-document corpus with 20 documents containing “analyt book”:

$$tf - idf = \frac{\text{count of term within document}}{\text{total words within document}} \times \log \frac{\text{total documents within corpus}}{\text{documents containing the term}}$$

- Count of “analyt book” in the document: 1
- Total words in the document: 9
- Total documents in the corpus: 100
- Documents containing the term: 20

$$tf - idf = (1/9) \times \log(100/20) = 0.1 \times 0.7 = 0.07$$

This TF-IDF value (0.07) is specific to this document — the same term would likely score differently elsewhere, since TF varies by document. Higher TF-IDF values indicate greater relative importance; in practice, only words above a chosen threshold (or the top-scoring subset) are kept as features for downstream modeling. Since IDF is constant for a term across a fixed corpus, it can be precomputed and reused efficiently.

Table 11.3-1: TF-IDF values for an example document set

Document	bark	big	cat	dog	see
"A dog barks to show a big dog"	0.04	0.00	0.00	0.09	0.00
"A big cat sees a barking dog"	0.04	0.00	0.00	0.04	0.04
"Some cats see some big cats"	0.00	0.00	0.24	0.00	0.04

Sentiment Analysis from Text

Sentiment analysis needs text as the input variable and a labeled sentiment as the target, as in this example dataset from the PyThaiNLP project, with text and sentiment (pos/neg) columns, roughly balanced across 128 examples:

Table 11.3-2: Example sentiment-analysis dataset from PyThaiNLP

text	sentiment
Good quality product, sturdy, good sound, kids love it, and fast shipping too	pos
Price matches quality, easy to use	pos
Fast delivery, good packaging, excellent product	pos
Used in the car, system broke, unusable	neg
Poor quality product, pant legs uneven	neg
More expensive than market price, terrible, a waste of money	neg
...	...

(Original text in Thai; translated here for illustration.)

Using PyThaiNLP for Thai tokenization and Scikit-Learn for modeling:

```
import numpy as np
import pandas as pd
from pythainlp import word_tokenize
from pythainlp.corpus.common import thai_stopwords
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_validate

CSV_PATH = "https://rathachai.github.io/DA-LAB/datasets/review-
shopping.csv"
df = pd.read_csv(CSV_PATH)
tfidf_vec = TfidfVectorizer(
    tokenizer=word_tokenize, stop_words=list(thai_stopwords()))
X = tfidf_vec.fit_transform(df['text']).toarray()
y = df['sentiment']
model = LogisticRegression()
scores = cross_validate(model, X, y, scoring='accuracy', cv=5,
    return_estimator=True)
print('accuracy = ', scores['test_score'].mean())

accuracy = 0.7883076923076924
```

Once satisfied with the model's performance, retrain it on the full dataset for real-world use, then try it on new text:

```

model = LogisticRegression()
model.fit(X, y)
new_texts = ['Beautifully made', 'Product arrived broken, nothing good
about it']
new_X = tfidf_vec.transform(new_texts)
print(model.predict(new_X))

['pos' 'neg']

```

(Example Thai text translated for illustration.)

Once a suitable technique is chosen, the trained model and `tfidf_vec` object can be saved for use in a real application. Text analytics has advanced significantly; readers are encouraged to explore deep-learning approaches such as Word2Vec and Transformers for further study.

11.4 Image Data Analytics Techniques

Image data analytics processes images to answer questions or uncover insight about their content — for example, detecting objects (faces, vehicles, trees) for uses like facial recognition, hazard detection, or species identification, or classifying images by content (portrait, animal, building). This has value across industries — automotive, healthcare, security, entertainment — improving operations and adding business value.

This section covers the nature of image data, deep learning for image classification, and a worked example.

Image Data

Human vision works by light reflecting off objects into the eye, forming an image on the retina, which is relayed via the optic nerve to the brain for interpretation.

For a computer, an image is represented as a matrix, where each element is a pixel's brightness value. A 480×640-pixel image is a 480×640 matrix, with each value ranging 0–255 (0 = black, 255 = white).

Figure 11.4-1 illustrates this: (1) a full book image; (2) zooming in reveals square pixel blocks; (3) these correspond to a grayscale matrix of values. Representing color (RGB) would use three such matrices, one per color channel.

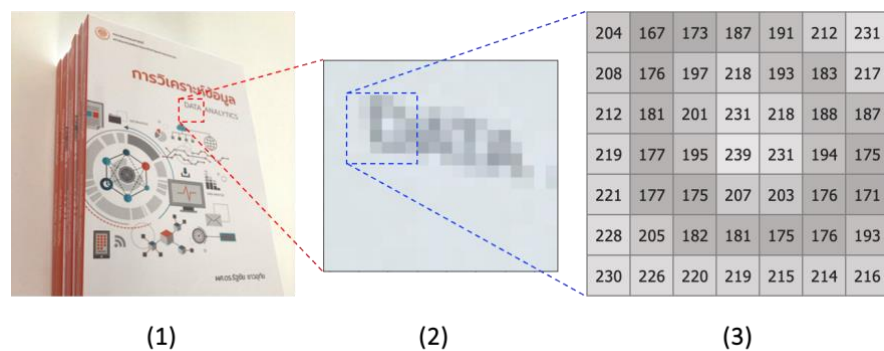


Figure 11.4-1: How a computer sees an image

Deep Learning for Image Data

Since images as matrices are complex, basic machine-learning techniques from earlier chapters don't work well directly, calling for the popular advanced technique of deep learning — specifically, Convolutional Neural Networks (CNN), briefly introduced here as a foundation for further study.

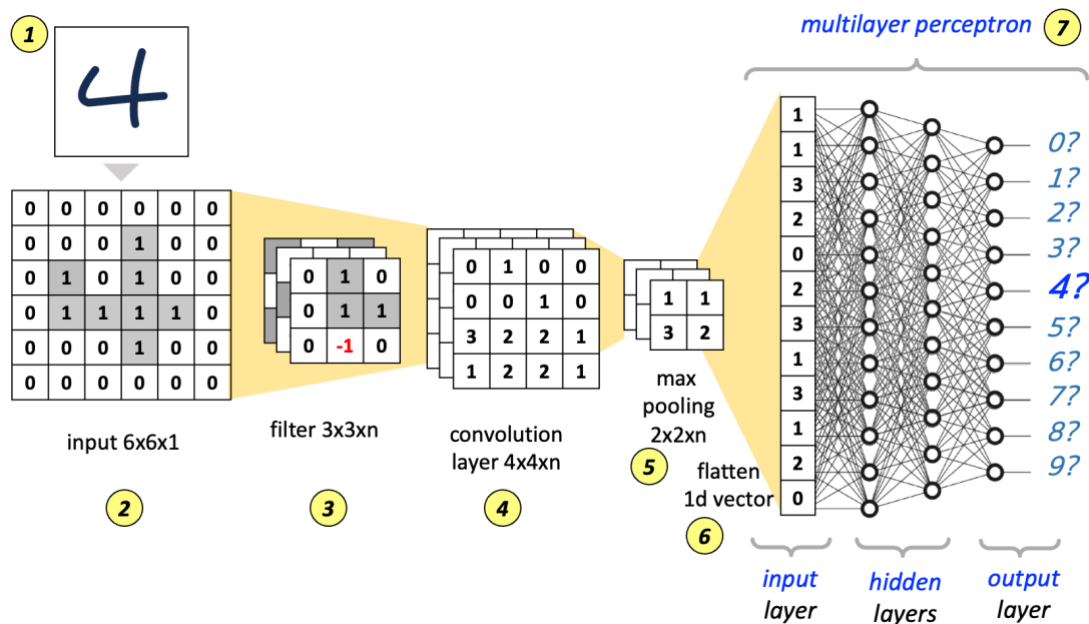


Figure 11.4-2: Using CNN for image classification

CNNs are deep-learning models purpose-built for image data, using filters to extract distinctive features of objects in an image. The technique involves several layers (Figure 11.4-2), each with many filters: early layers process basic information (pixel brightness), middle layers process higher-level shapes, and final layers process the highest-level information — the object's category. Each stage is explained below.

Input

The input is an image represented as a matrix, as described above. For simplicity, this example uses a single grayscale matrix (readers can later adapt this to RGB). The example converts an image of the digit “4” (step 1) into a 6×6 grayscale matrix (step 2).

Filter

A CNN filter is a set of constants used to filter image data, extracting distinctive features like shape, edges, or orientation. The example (step 3) uses three 3×3 filters, chosen by the developer as a starting point; the actual filter values are learned and refined by the training algorithm to maximize model accuracy.

Convolution Layer

The convolution layer (step 4) results from processing the image matrix against the filter matrix, passed through the Rectified Linear Unit (ReLU) function, popular in deep learning — linear when the input is ≥ 0 , and zero otherwise:

$$ReLU(x) = \max(x, 0)$$

The calculation selects a filter-sized region of the image matrix (e.g., 3×3), multiplies it against the filter, and records the result in the convolution layer, then slides across the image matrix one step at a time until fully covered. Figure 11.4-3 shows the first such 3×3 calculation.

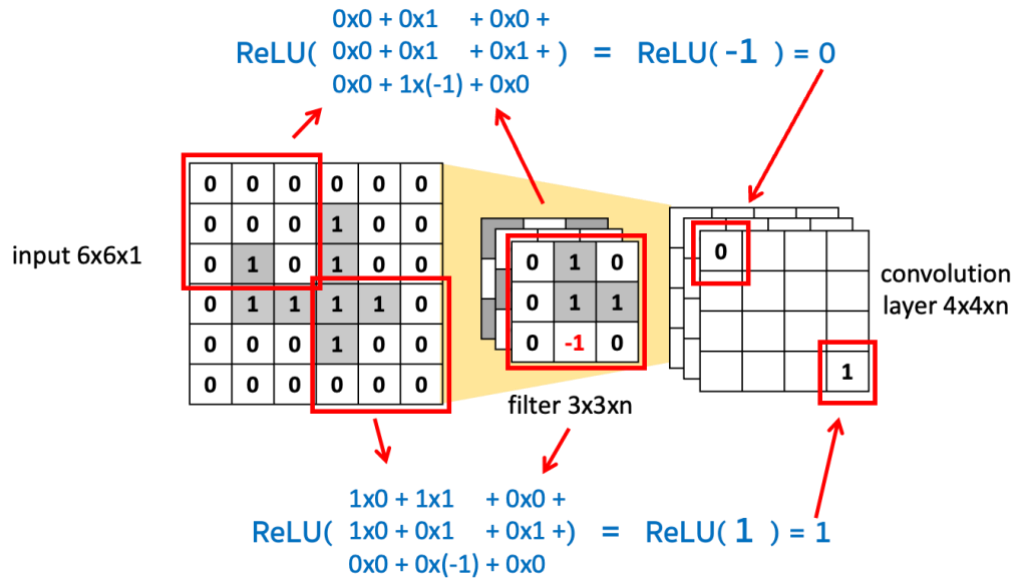


Figure 11.4-3: Example convolution-layer calculation

Pooling Layer

The pooling layer (step 5) typically uses max pooling to shrink the data while retaining key information, taking the maximum value from each adjacent pixel group of a chosen size — e.g., 2×2 means taking the max of each 2×2 block of the convolution layer, sliding 2 steps at a time across the whole matrix, as in Figure 11.4-4.

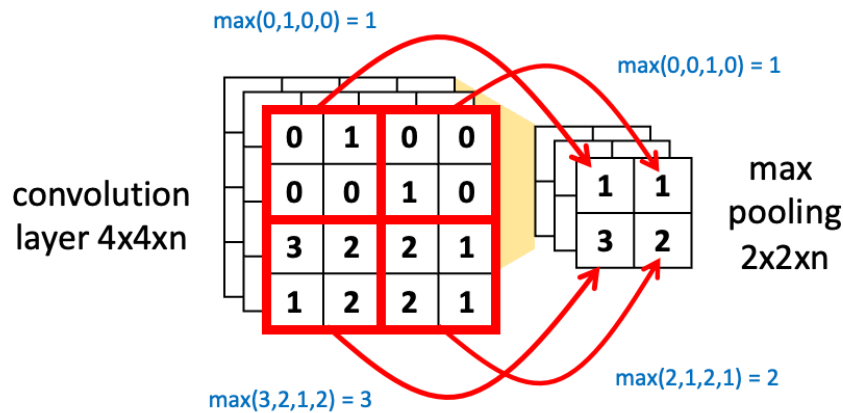


Figure 11.4-4: Example max-pooling calculation

Flatten Layer

Pooling produces several smaller matrices, depending on the number of filters used earlier. Before classification, all these matrices must be combined into a single feature vector (step 6), sized to the total number of elements across all matrices.

Note: at this stage, dropping some data — Dropout — is often recommended, a regularization technique that prevents the model from overfitting to overly complex patterns during training, which would otherwise hurt test-time prediction. In practice, 20–25% of data is commonly dropped, at the developer's discretion.

Dense Layer

With features from the flatten layer, the next step builds a neural network (Dense Layer) for classification, per the Artificial Neural Network technique in section 7.4.5. The flattened vector serves as the input layer; developers choose the number of hidden layers, nodes per layer, and activation function, with an output layer sized to the number of classes. Figure 11.4-2's example uses 2 hidden layers, with 12 and 11 nodes respectively.

Image Classification with CNN in TensorFlow

This example demonstrates simple CNN-based image classification using Keras (TensorFlow), adapted from:

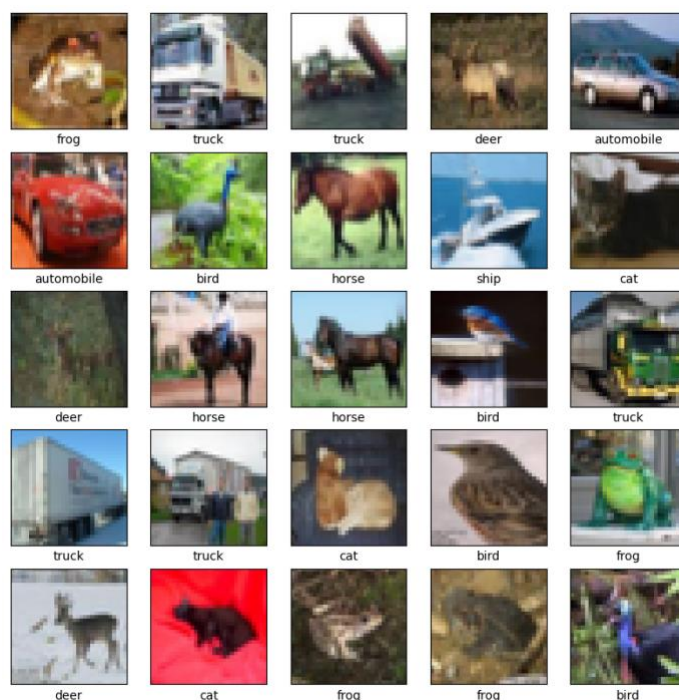
<https://www.tensorflow.org/tutorials/images/cnn>

using the CIFAR10 dataset: 60,000 color images across 10 categories — automobile, truck, airplane, ship, dog, cat, deer, horse, frog, and bird — each 32×32 pixels.

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
```

```
(train_images, train_labels), (test_images, test_labels) =
datasets.cifar10.load_data()
train_images = train_images / 255.0
test_images = test_images / 255.0
```

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i])
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```



```

model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))
model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36928
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65600
dense_1 (Dense)	(None, 10)	650
Total params: 122570 (478.79 KB)		
Trainable params: 122570 (478.79 KB)		
Non-trainable params: 0 (0.00 Byte)		

```

model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
model.fit(train_images, train_labels, epochs=10,
          validation_data=(test_images, test_labels))

```

```

Epoch 1/10
1563/1563 [=====] - 13s 5ms/step - loss: 1.5360 - accuracy: 0.4374 - val_loss: 1.2494 - val_accuracy: 0.5548
Epoch 2/10
1563/1563 [=====] - 8s 5ms/step - loss: 1.1629 - accuracy: 0.5865 - val_loss: 1.1343 - val_accuracy: 0.6001
Epoch 3/10
1563/1563 [=====] - 7s 5ms/step - loss: 0.9928 - accuracy: 0.6491 - val_loss: 0.9582 - val_accuracy: 0.6642
Epoch 4/10
1563/1563 [=====] - 8s 5ms/step - loss: 0.8941 - accuracy: 0.6844 - val_loss: 0.9154 - val_accuracy: 0.6793
Epoch 5/10
1563/1563 [=====] - 7s 5ms/step - loss: 0.8222 - accuracy: 0.7129 - val_loss: 0.8942 - val_accuracy: 0.6916
Epoch 6/10
1563/1563 [=====] - 7s 4ms/step - loss: 0.7644 - accuracy: 0.7337 - val_loss: 0.9235 - val_accuracy: 0.6822
Epoch 7/10
1563/1563 [=====] - 8s 5ms/step - loss: 0.7088 - accuracy: 0.7516 - val_loss: 0.8631 - val_accuracy: 0.7017
Epoch 8/10
1563/1563 [=====] - 7s 4ms/step - loss: 0.6647 - accuracy: 0.7674 - val_loss: 0.8772 - val_accuracy: 0.7078
Epoch 9/10
1563/1563 [=====] - 8s 5ms/step - loss: 0.6194 - accuracy: 0.7792 - val_loss: 0.8902 - val_accuracy: 0.7049
Epoch 10/10
1563/1563 [=====] - 7s 4ms/step - loss: 0.5797 - accuracy: 0.7954 - val_loss: 0.8760 - val_accuracy: 0.7155

```

```

test_loss, test_acc = model.evaluate(test_images, test_labels)
print("Accuracy = ", test_acc)

```

Accuracy = 0.7113000154495239

This is only a brief introduction to deep learning for image data; real-world work involves far more complexity depending on the problem. Readers are encouraged to study computer vision from fundamentals through advanced AI techniques for building more effective models.

11.5 Chapter Summary

This chapter rounded out the book with additional techniques for further study: SMOTE for imbalanced data, PCA for dimensionality reduction, key techniques for text data analytics, and CNNs for image data analytics. Working with unstructured data is often considerably more complex depending on the specific problem, so readers are encouraged to keep following new techniques and improvements in this space, valuable for both research and business applications.

11.6 Review Questions

1. Find 5 real-world examples of imbalanced datasets.
2. Using the Iris dataset from <https://rathachai.github.io/DA-LAB/datasets/iris.csv>, take the first 60 rows (60 setosa examples and only 10 versicolor examples), then balance it using SMOTE.
3. From question 2, compare classification model performance (same technique) on the data before and after balancing, using 3-fold cross-validation and reporting accuracy.
4. Discuss whether synthetically generated minority-class data (e.g., from SMOTE) should be included in the test set, and why or why not.
5. Using the Iris dataset code from section 11.2.1, reduce it to 2 dimensions with PCA, then classify it using techniques from earlier chapters, reporting each model's performance.
6. Study the mall customer behavior dataset from Kaggle, referenced at <https://github.com/Rathachai/DA-LAB/tree/gh-pages/exercises> ("Mall Customer Behavior Dataset"), then write Python code to cluster it with DBSCAN and visualize the result in 2D using PCA.
7. Using the data from question 6, cluster with K-Means on both the original data and the PCA-reduced (2D) data, and explore whether the optimal K differs between the two.
8. Using <https://rathachai.github.io/DA-LAB/datasets/text-topic.csv> (text data with topic labels), write a program to: (a) tokenize the "text" column, (b) remove stop words, (c) apply stemming, (d) compute TF-IDF for each word in each example, (e) keep only the top 10 TF-IDF words per example, and (f) build a topic-classification model, evaluating with 4-fold cross-validation and reporting F1.
9. From the "4" digit matrix in Figure 11.4-2 (step 2): (a) compute the convolution result using the filter $\begin{bmatrix} 1 & -1 \\ 1 & 0 \end{bmatrix}$ passed through ReLU; (b) compute the result of a 2×2 max-pooling layer.
10. Load the CIFAR100 image dataset via `keras.datasets.cifar100.load_data()`, then build a CNN classification model achieving accuracy above 0.8.

11.7 References

- Gupta, H., Kaur, A., Kavita, Verma, S., & Rawat, P. (2023, February). Recognition of Handwritten Digits Using Convolutional Neural Network in Python and Comparison of Performance for Various Hidden Layers. In International Conference on Innovative Computing and Communication (pp. 727–739). Springer Nature Singapore.
- Hasan, B. M. S., & Abdulazeez, A. M. (2021). A Review of Principal Component Analysis Algorithm for Dimensionality Reduction. *Journal of Soft Computing and Data Mining*, 2(1), 20–30.
- Jansson, N. F., Allen, R. L., Skogsmo, G., & Tavakoli, S. (2022). Principal Component Analysis and K-Means Clustering as Tools During Exploration for Zn Skarn Deposits and Industrial Carbonates, Sala Area, Sweden. *Journal of Geochemical Exploration*, 233, 106909.
- Krichen, M. (2023). Convolutional Neural Networks: A Survey. *Computers*, 12(8), 151.
- Lee, R. S. (2023). *Natural Language Processing: A Textbook with Python Implementation*. Springer Nature.

Lowphansirikul, C. P. A. S. L., Phatthiyaphaibun, P. C. W., & Chaovavanich, K. (2016). PyThaiNLP: Thai Natural Language Processing in Python.

Nirthika, R., Manivannan, S., Ramanan, A., & Wang, R. (2022). Pooling in Convolutional Neural Networks for Medical Image Analysis: A Survey and an Empirical Study. *Neural Computing and Applications*, 34(7), 5321–5347.

Phatthiyaphaibun, W., Chaovavanich, K., Polpanumas, C., Suriyawongkul, A., Lowphansirikul, L., Chormai, P., et al. (2023, October). PyThaiNLP: Thai Natural Language Processing in Python. In 3rd Workshop for Natural Language Processing Open Source Software.

Sharma, S., Gosain, A., & Jain, S. (2022). A Review of the Oversampling Techniques in Class Imbalance Problem. In *International Conference on Innovative Computing and Communications: Proceedings of ICICC 2021*, Volume 1 (pp. 459–472). Springer Singapore.

Wright, J., & Ma, Y. (2022). *High-Dimensional Data Analysis with Low-Dimensional Models: Principles, Computation, and Applications*. Cambridge University Press.

Wongvorachan, T., He, S., & Bulut, O. (2023). A Comparison of Undersampling, Oversampling, and SMOTE Methods for Dealing with Imbalanced Classification in Educational Data Mining. *Information*, 14(1), 54.